

Containerization & Podman

Siddhesh Prabhugaonkar

Microsoft Certified Trainer

siddheshpg@azureauthority.in

Learning goals

- Understand *why* containerization solves modern deployment problems vs traditional approaches and VMs.
- Learn core container concepts and Dockerfile instructions.
- Use **Podman Desktop** on Windows/macOS to build images, run containers, and push images to a registry.
- Compose the EMS system (React, NestJS, Oracle DB) with **podman-compose** and run it locally.
- Apply Dockerfile best practices and container security/volume considerations.

1) Why containerization?

Problems with traditional deployments

- “Works on my machine” due to mismatched runtimes, libraries, OS dependencies.
- Heavy resource footprint for full VMs; slow boot times.
- Manual configuration drift across environments.

Benefits of containerization

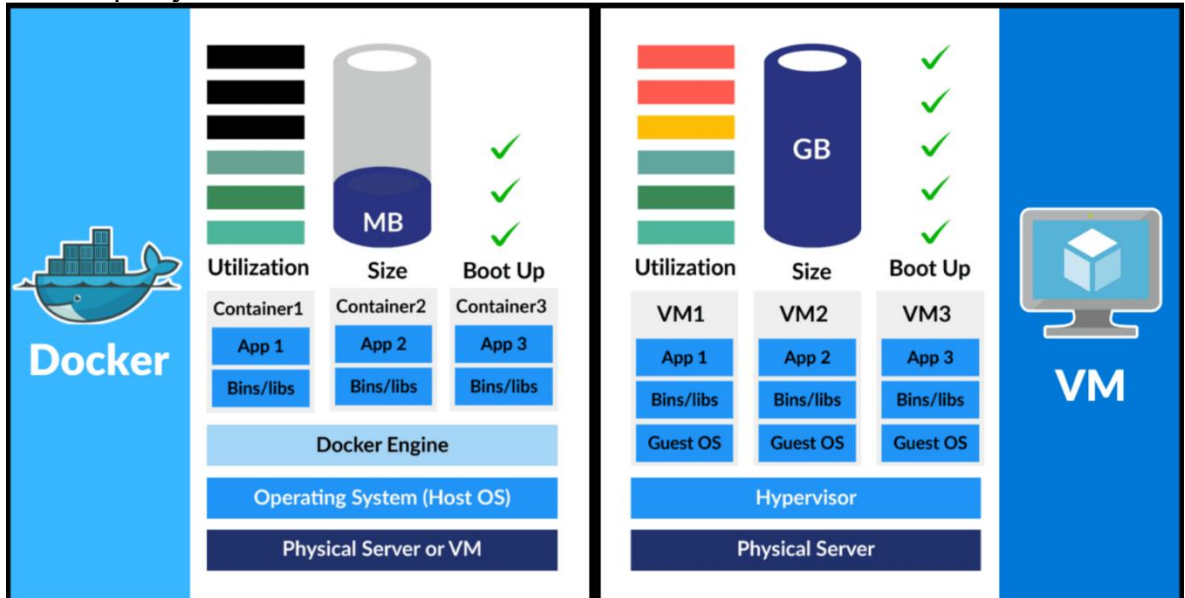
- Lightweight, consistent environments packaged with app + dependencies.
- Fast startup, efficient density on hosts.
- Portability across environments (dev → CI → prod).
- Easier CI/CD, scaling, and microservice decomposition.

A quick conceptual diagram (architecture):

```
[Host OS]
├── [VM approach] -> hypervisor -> guest OS -> app (heavy)
└── [Container approach] -> container runtime -> containers -> app (light)
```

2) Containers vs VMs

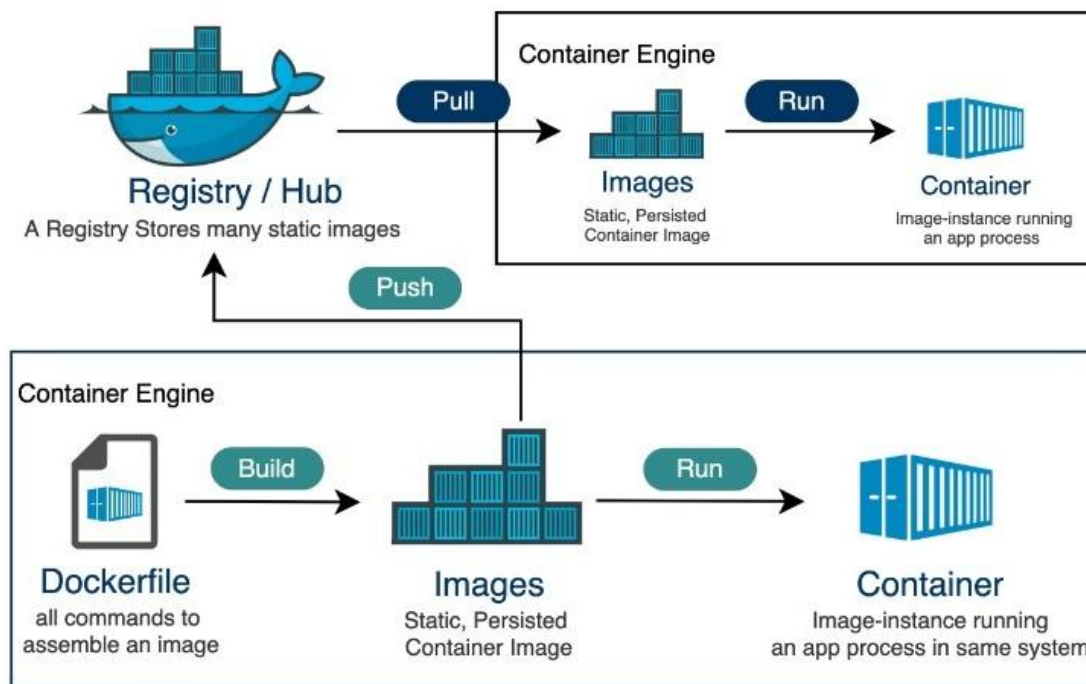
- **Isolation:** VMs include a full guest OS + kernel; containers share host kernel (lighter).
- **Resource overhead:** VMs heavier (CPU/memory); containers small and fast to start.
- **Use cases:** VMs for OS-level isolation/compliance; containers for microservices and dev/test parity.



3) Docker / Container engine basics (Podman context)

- **Engine / Runtime:** Docker Engine vs Podman engine — both run containers; Podman is daemonless and can run rootless by default.
- **CLI:** Most `docker` commands have Podman equivalents (e.g., `podman run`, `podman build`, `podman push`).
- **Images vs containers:** Image = snapshot; Container = running instance of an image.

Podman Desktop is cross-platform (Windows/macOS) GUI + tooling for managing containers. podman-desktop.io

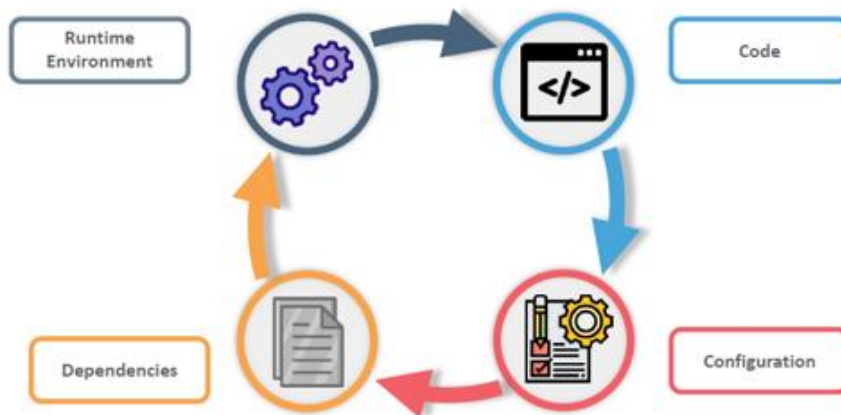


4) Core Dockerfile instructions

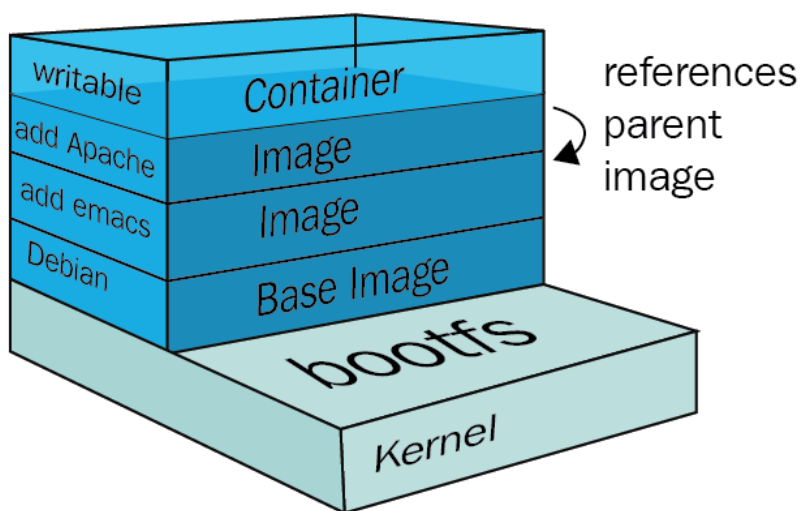
- `FROM <image>` — base image; always pin versions when possible. Example: `FROM node:20-alpine`.
- `LABEL` — metadata.
- `ENV KEY=value` — environment variables baked into image.
- `WORKDIR /app` — change working directory.
- `COPY <src> <dest> / ADD` — copy files into image. Prefer `COPY`; use `.dockerignore`.
- `RUN <cmd>` — execute at build time (creates layer).
- `EXPOSE <port>` — documents the port; doesn't publish by itself.
- `CMD ["node", "index.js"]` — default process to run (overridable).
- `ENTRYPOINT` — makes a container behave like an executable.
- **Multi-stage builds** — produce small final images (build stage → runtime stage).

Best practices: minimize layers, pin base images, prefer small base images (alpine), use `.dockerignore`, keep secrets out of Dockerfile, and use multi-stage builds for compiled apps.

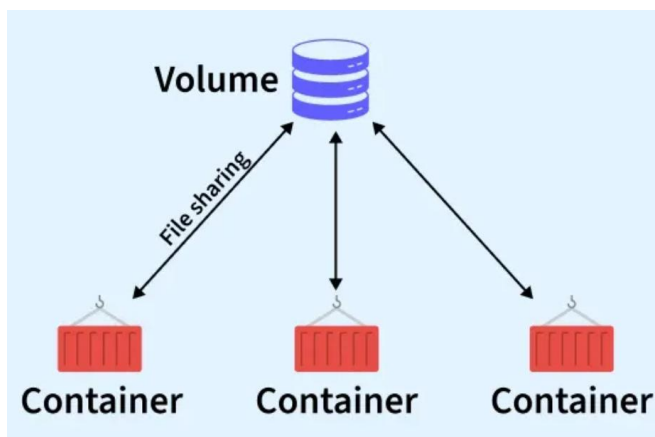
Docker Image content



Docker image layers



Volume



5) EMS case study design

Three containers:

1. **React frontend** — built into static bundle; served by `nginx` or `http-server`.
2. **NestJS API** — Node app exposing REST endpoints.
3. **Oracle DB** — Oracle XE (use a supported image or build from Oracle-provided Dockerfiles due to licensing).

Network: containers exposed on internal compose network; ports forwarded for local dev (frontend 5173, API 3000, Oracle 1521). Use volumes for Oracle persistence.

Oracle images are available via Oracle container registry or community images (confirm license/terms before production).

6) Podman Compose

`podman-compose` (or `podman compose`) is used to run multi-container apps from a `docker-compose.yml`-style file; native Podman supports `podman compose` and multiple external compose implementations exist. **Podman's approach is daemonless**; some compose behavior differences exist vs Docker Compose — test your compose files.

Podman Compose lets you run **multiple containers together** as a single application.

Instead of running containers separately like:

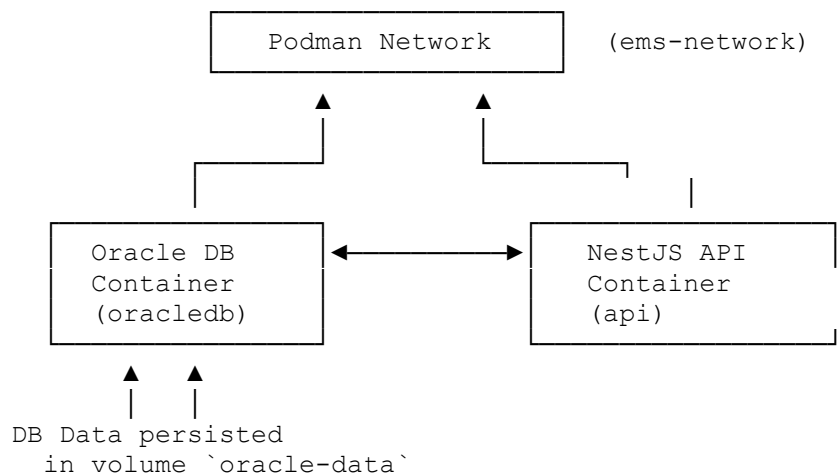
- `podman run oracle`
- `podman run api`

...you define everything in **one YAML file** (`podman-compose.yml`) and simply run:

```
podman compose up
```

This:

- Creates containers
- Connects them on a common network
- Injects environment variables
- Manages startup order
- Sets volumes, ports, restart policies, health checks, etc.



Example podman-compose.yml

```

version: '3.8'
# Specifies the Compose file format version

services:
  # -----
  # ORACLE DATABASE SERVICE
  # -----
  oracledb:
    image: container-registry.oracle.com/database/express:21.3.0-xe
    # Pull and run the official Oracle XE 21c database image

    container_name: ems-oracledb
    # Easy-to-read name when running podman commands

    ports:
      - "1521:1521" # Expose Oracle DB port to host so API/DB tools can connect
      - "5500:5500" # Expose Oracle EM Express (web UI) on host machine

    environment:
      - ORACLE_PWD=OraclePassword123
      # Sets initial SYS/SYSTEM admin password inside the DB container

    volumes:
      - oracle-data:/opt/oracle/oradata
      # Persists actual Oracle database files so data isn't lost on container restart

      - ./oracle-init:/opt/oracle/scripts/startup
      # Any SQL scripts inside ./oracle-init run automatically on DB startup

    healthcheck:
      # A way to test if DB is up, running, and accepting SQL connections
      test: ["CMD", "sqlplus", "-L",
        "sys/OraclePassword123@//localhost:1521/XE as sysdba", "@/dev/null"]
      interval: 30s # Run the check every 30 seconds
      timeout: 10s # Mark as failed if it takes longer than 10 seconds
  
```

```

    retries: 5          # Try 5 times before marking container as unhealthy

networks:
  - ems-network
  # Ensures DB can communicate with other services on this private
virtual network

# -----
# NESTJS API SERVICE
# -----
api:
  build:
    context: .
    # Build image from current folder (should contain Dockerfile)

    dockerfile: Dockerfile
    # Dockerfile used for building backend image

  container_name: ems-backend
  # Easy name for the API container

  ports:
    - "3000:3000"
    # Exposes API on host machine at http://localhost:3000

  environment:
    # -----
    # OKTA CONFIG (pulled from your .env)
    # -----
    - OKTA_ISSUER=${OKTA_ISSUER}
    - OKTA_CLIENT_ID=${OKTA_CLIENT_ID}
    - OKTA_AUDIENCE=${OKTA_AUDIENCE}

    # -----
    # DATABASE CONNECTION CONFIG
    # -----
    - DB_TYPE=oracle
    - DB_HOST=oracledb          # IMPORTANT: service name, not localhost
    - DB_PORT=1521
    - DB_USERNAME=training
    - DB_PASSWORD=training123
    - DB_SERVICE_NAME=XEPDB1   # Oracle PDB name
    - DB_SYNCHRONIZE=true      # Auto-create tables (dev only)

  depends_on:
    oracledb:
      condition: service_healthy
      # API waits until Oracle DB passes its healthcheck.
      # Prevents "database not ready" errors.

  networks:
    - ems-network
    # API joins same network so it can talk to 'oracledb' container by
name

  restart: unless-stopped
  # Automatically restarts API unless user explicitly stops it

# -----

```

```
# VOLUMES SECTION
# -----
volumes:
  oracle-data:
    driver: local
    # Defines named volume used for Oracle data files (persistent)

# -----
# NETWORK SECTION
# -----
networks:
  ems-network:
    driver: bridge
    # Creates shared internal network so API ↔ DB can communicate by
    service name
```

Run with:

- `podman compose up --build` or `podman-compose up --build` depending on your install.

Notes: if you use `podman` push to Docker Hub, tag images like `docker.io/<user>/<image>:tag`.

7) Hands-on labs — Windows and macOS

Pre-reqs (both)

- Install Podman Desktop (Windows/macOS installers available).
- Install `podman` CLI (Podman Desktop includes it).
- Install `podman-compose` (if needed) or ensure `podman compose` is available.

Lab A — Build & run NestJS API image (both OS)

1. Open terminal (PowerShell on Windows / Terminal on macOS).
2. In `ems-api` source folder (assumed Node project):
 - o Add `Dockerfile` (example below).
3. Build: `podman build -t <dockerhub-user>/ems-api:dev .`
4. Run: `podman run --rm -p 3001:3001 --name ems-api -e DB_HOST=oracle <dockerhub-user>/ems-api:dev`
5. Exec into container: `podman exec -it ems-api /bin/sh (or bash)`.

Notes for Windows: Run PowerShell as Administrator only if you need host port binding on privileged ports. Podman Desktop config may require enabling WSL backend (Windows).

Lab B — Build frontend & serve (both OS)

1. In `ems-frontend`, build React production bundle locally: `npm run build`.
 2. Dockerfile stage to copy build into `nginx` or `http-server` image. Build with `podman build -t <user>/ems-frontend:dev ..`
 3. Run `podman run -d -p 3000:80 <user>/ems-frontend:dev`.
-

Lab C — Oracle DB container

1. Pull or use Oracle XE image: `podman pull gvenzl/oracle-xe:23-slim` (or use Oracle's registry following their quickstart).
2. Run persistent container:
3. `podman run -d --name oracle -p 1521:1521 -v oracle-data:/opt/oracle/oradata \`
4. `-e ORACLE_PASSWORD=oracle gvenzl/oracle-xe:23-slim`
5. Confirm DB up via `tnsping` / or connect via your app.

(Confirm license acceptance; production use requires following Oracle licensing guidelines.)

8) Example Dockerfile (Node / NestJS) — annotated

```
# build stage
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --production=false
COPY . .
RUN npm run build

# runtime stage
FROM node:20-alpine AS runner
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY package*.json ./
RUN npm ci --production=true
ENV NODE_ENV=production
CMD ["node", "dist/main.js"]
EXPOSE 3001
```

Why multi-stage: keeps final image small by excluding build-time deps. Use `--production=true` in final stage to only install runtime deps. Follow Docker build caching best practices (`COPY package*.json` before source to leverage cache).

9) Best practices & security notes

- **Pin base images** (node:20-alpine -> node:20.5.0-alpine if needed).
- Use `.dockerignore`.
- Do not bake secrets into images — use env vars or secret managers.
- Use non-root users inside images when possible.
- Keep images small and multi-stage builds where appropriate.
- For production, prefer orchestration (Kubernetes) and consider image scanning.

Dockerfile/build best practices references.

10) Common questions & quick answers

Q: Can I push images built with Podman to Docker Hub?

A: Yes — `podman push docker://docker.io/<user>/<image>:tag` after `podman login docker.io`. Podman supports Docker Hub and other registries.

Q: Is podman-compose fully compatible with Docker Compose?

A: Mostly, but there are edge cases — test your compose file. In many setups `podman compose` (the integrated implementation) works well; `podman-compose` is a separate project with some differences.

Q: Which Oracle image should I use?

A: Oracle provides images via Oracle Container Registry and GitHub `docker-images`; community images (e.g., `gvenzl/oracle-xe`) exist — always check license/production suitability.

11) Student exercise (practical)

- Fork a repo with `ems-api`, `ems-frontend` folders.
 - Create Dockerfiles for both.
 - Create `podman-compose.yml` as above and run `podman compose up --build`.
 - Add health checks and a volume-backed Oracle instance.
 - Push built images to Docker Hub using `podman push`.
-

12) Consolidated commands

Podman / build / run / compose / push (cross-platform)

```
# Install - follow Podman Desktop installer for Windows/macOS (GUI + CLI)
# Build images
podman build -t docker.io/<user>/ems-api:dev ./ems-api
podman build -t docker.io/<user>/ems-frontend:dev ./ems-frontend

# Run single containers
podman run --rm -d --name oracle -p 1521:1521 -v oracle-
data:/opt/oracle/oradata \
  -e ORACLE_PASSWORD=oracle gvenzl/oracle-xe:23-slim

podman run --rm -d --name ems-api -p 3001:3001 \
  -e DB_HOST=oracle docker.io/<user>/ems-api:dev

podman run --rm -d --name ems-frontend -p 3000:80 docker.io/<user>/ems-
frontend:dev

# Compose (build + up)
podman compose up --build
# or if using separate project
podman-compose up --build

# Login and push to Docker Hub
podman login docker.io
podman push docker.io/<user>/ems-api:dev
podman push docker.io/<user>/ems-frontend:dev

# Useful inspection commands
podman ps -a
podman logs <container>
podman exec -it <container> /bin/sh
podman images
podman stop <container>
podman rm <container>
```

Podman Run Parameters

Parameter	Meaning / Usage
-d	Run container in detached (background) mode
-i	Keep STDIN open (interactive)
-t	Allocate a TTY (for terminal access)
--rm	Auto-remove container when it exits
--name <name>	Assign a custom container name
-p host:container	Map ports from container to host
-e KEY=value / --env KEY=value	Set environment variables

Parameter	Meaning / Usage
--env-file <file>	Load env variables from a file
-v host:container / --volume	Mount a host path/volume into container
--pull=always/missing/never	Control image pull behavior
--restart=always/on-failure/no	Configure container restart policy
--network=<mode>	Set container network mode (bridge, host, none)
--hostname <name>	Set the container's hostname